DEPARTMENT OF COMPUTER SCIENCE AND INFORMATICS,
MIDDLESEX UNIVERSITY

PDE4444 - MACHINE LEARNING FOR ENGINEERS

Convolutional Neural Network based PCB Orientation Verification relative to Surface Reference

Prepared by : D. H. Vanderbona (M01088317)
: A. S. Shaikh (M01096196)
Professor : Dr. Maha Saadeh
Date : 24/4/2026

ABSTRACT

Printed Circuit Board (PCB) orientation verification is a critical step in automated manufacturing and quality assurance. Traditional inspection systems rely on fixed-position cameras and mechanical alignment fixtures, which require precise calibration and are prone to degradation over time. To address these limitations, this work presents a machine learning–based inspection framework that classifies PCB orientation relative to a reference line, producing a binary PASS/FAIL output. Two complementary approaches were implemented: a Convolutional Neural Network (CNN) as the primary model and a Histogram of Oriented Gradients (HOG) feature extractor with a Support Vector Machine (SVM) classifier as the baseline. The CNN achieved superior performance, with validation accuracy of 96.40%, mean cross-validation accuracy of 93.43% $\pm$ 0.16%, and F1-score of 92.26%, compared to the SVM baseline at 85.48% validation accuracy and 80.61% F1-score. The dataset was constructed from video recordings under varying angles and lighting conditions, ensuring balanced PASS/FAIL representation. Results demonstrate that robust orientation detection can be achieved using a simple camera system, with CNN providing excellent generalization and resilience to positional variability. This study underscores the trade-off between flexibility in camera placement and classification accuracy, while highlighting the industrial impact: improved defect detection, reduced reliance on manual inspection, and scalable, low-cost deployment across PCB manufacturing environments.

Contents

1 Introduction **1**

1.1 Introduction . . . 1

1.2 Problem Statement . . . 1

1.3 Proposed Solution . . . 2

1.3.1 Workflow . . . 2

1.3.2 Model Justification . . . 3

2 Methodology **4**

2.1 Model Architecture . . . 4

2.1.1 Baseline Architecture: HOG + SVM . . . 5

2.2 Dataset Accumulation . . . 5

2.2.1 Train, Validation and Test Split . . . 6

2.2.2 Post-Inference Probability Calibration for Imbalanced Data . . . 7

2.3 Feature Representation . . . 7

2.4 Optimisation . . . 8

2.4.1 Activation Function Selection . . . 8

2.4.2 Hyperparameter Tuning . . . 9

2.4.3 Cross-Validation . . . 9

2.4.4 Evaluation Metrics . . . 10

2.4.5 Outcome . . . 10

2.4.6 Overfitting Analysis . . . 11

3 Testing and Results **13**

3.1 CNN vs. HOG+SVM Baseline Comparison . . . 13

3.1.1 Accuracy Comparison . . . 13

3.1.2 Precision, Recall, and F1-Score . . . 13

3.1.3 Generalization Analysis . . . 13

3.1.4 Key Differences Analysis . . . 14

3.1.5 Model Selection . . . 14

3.2 Test Configuration . . . 15

3.3 Results . . . 15

3.4 Failure Cases and Limitations . . . 16

4 Recommendations and Future Enhancements **18**

4.1 Immediate Actions . . . 18

4.2 Mid-Term Enhancements (Months 1–3) 18
4.3 Long-Term Strategy (Months 3+) 18

References **20**

Appendix **21**

List of Figures

1.3.1 Target orientation relative to a reference line 2

2.2.1 Sample Pass Image 6
2.2.2 Sample Fail Image 6
2.4.1 Convergence comparison of ReLU, ELU, and Sigmoid activation functions. 9

3.1.1 ROC curve comparison between CNN and SVM models. 14
3.3.1 CNN Confusion matrix for 500-image test set. 15
3.3.2 HOG + SVM Confusion matrix for 500-image test set. 16
3.4.1 Instance of output instability due to motion blur 17
3.4.2 Instance of output instability due to gap in working envelope 17

4.3.1 Folder structure of the dataset used for PCB orientation verification. 21
4.3.2 Default image extractor config (paths may have change) 21
4.3.3 Sample Fail and Pass images from live camera classification 22
4.3.4 Sample Images manually removed from the data set 23

List of Tables

2.1 Cross-validation overfitting check 10

3.1 Accuracy comparison between CNN and SVM 13
3.2 Performance metrics comparison 13
3.3 Comparison of CNN and SVM characteristics 14

Chapter 1
Introduction

1.1 Introduction

Accurate orientation of Printed Circuit Boards (PCBs) is a critical requirement for reliable downstream assembly and testing operations. Even minor misalignments can lead to defective soldering, improper component placement, and increased rejection rates, ultimately reducing production efficiency and raising manufacturing costs. Conventional inspection systems typically rely on fixed-position cameras and mechanical alignment fixtures to ensure consistent imaging conditions. While effective, these systems demand precise calibration and ongoing maintenance, and any deviation in camera placement can compromise inspection accuracy.

Recent advances in computer vision and machine learning have enabled more flexible approaches to visual inspection. By training models on datasets that incorporate multiple viewing angles, it becomes possible to mitigate the effects of minor variations in camera positioning. However, despite this flexibility, the requirement for a stable imaging setup remains critical, as significant positional changes introduce distortions that degrade classification performance. This highlights the need for intelligent inspection systems that balance robustness with practical deployment constraints.

This paper proposes a supervised machine learning framework for PCB orientation verification, with a Convolutional Neural Network (CNN) as the primary model. The system evaluates PCB orientation relative to a simple surface reference line and outputs a binary *PASS/FAIL* classification, enabling automated detection of misaligned boards. The design emphasizes low hardware complexity by relying on a single camera system, while acknowledging practical constraints such as sensitivity to angular deviations, limited tolerance to translation, and the necessity of consistent camera placement within a defined working envelope. These limitations are addressed in later sections on optimisation and testing, where failure cases and robustness are discussed in detail.

1.2 Problem Statement

The challenge addressed in this paper is the reliable detection of PCB orientation using a low-cost camera system without reliance on complex mechanical fixtures. Traditional inspection methods assume that the camera remains perfectly fixed, and even minor deviations in position or angle can lead to erroneous classification. In practical manufacturing environments, maintaining such rigid constraints is difficult due to equipment wear, operator adjustments, or environmental factors such as vibrations. Consequently, there is a need for a machine learning-based solution that can tolerate limited variability in camera positioning while still ensuring accurate orientation assessment. The problem is therefore defined as designing a model that can classify PCB alignment against a reference

line under varying viewing angles, while acknowledging that excessive positional changes remain a limiting factor for performance. [1]

1.3 Proposed Solution

The proposed system introduces a simple yet robust method for PCB orientation verification relative to a fixed surface reference. A marked reference is placed on the conveyor belt (or any stable surface) to serve as the baseline orientation. A Convolutional Neural Network (CNN) is trained to detect the PCB's angular deviation with respect to this reference.

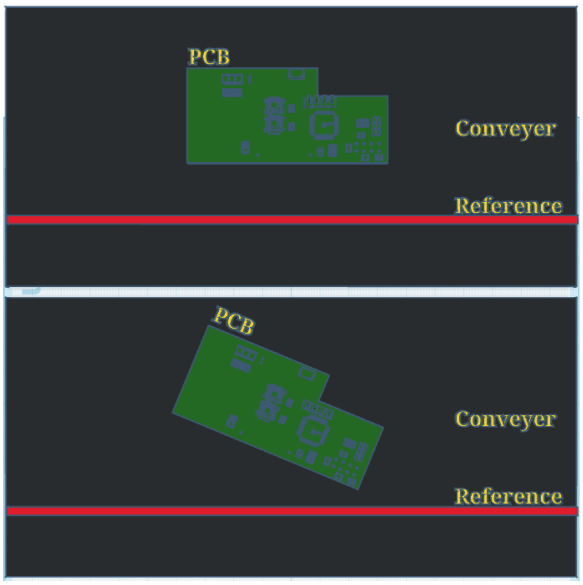

Figure 1.3.1: Target orientation relative to a reference line

1.3.1 Workflow

- A reference marker is embedded on the conveyor belt surface.
- The PCB is placed on the conveyor and imaged by a vision system.
- The CNN extracts orientation features of the PCB edges and compares them against the reference marker.
- The system outputs PASS or FAIL.

1.3.2 Model Justification

The selection of a Convolutional Neural Network (CNN) as the core model is motivated by its proven ability to capture spatial hierarchies in image data. PCB orientation verification requires discriminative features such as edges, line segments, and angular deviations relative to a reference marker. CNNs are particularly well-suited for this task because:

- **Spatial Feature Extraction:** Convolutional filters detect local geometric structures (edges, corners, line segments) that are critical for orientation analysis.
- **Hierarchical Representation:** Deeper layers progressively encode mid-level and high-level orientation cues, enabling robust classification of aligned versus misaligned boards.
- **Robustness to Variability:** Pooling and normalization operations provide resilience against minor variations in lighting, viewing angle, and PCB design.
- **Efficiency:** CNNs achieve high accuracy with relatively low computational cost compared to more complex architectures, making them suitable for deployment in real-time inspection systems.
- **Extensibility:** The modular design allows substitution with deeper networks if higher accuracy or deployment on constrained hardware is required.

Alternative models such as HOG feature extraction combined with an SVM classifier were considered as a baseline. While HOG+SVM provides consistent performance and interpretable linear decision boundaries, it relies on handcrafted features that are brittle under real-world variability. In contrast, CNNs automatically learn orientation-relevant features, achieving superior accuracy, precision, recall, and F1-score. This makes CNNs the optimal balance between accuracy, robustness, and scalability for PCB orientation verification, while HOG+SVM serves as a useful classical reference point.

Chapter 2

Methodology

The proposed system employs a supervised machine learning model to classify PCB orientation relative to a reference line. A Convolutional Neural Network (CNN) is selected as the baseline architecture due to its ability to capture spatial features in image data. The model is designed with three convolutional blocks (hidden layers), followed by dense layers for classification.

2.1 Model Architecture

The proposed Convolutional Neural Network (CNN) is designed to progressively learn orientation-specific features from PCB images while incorporating regularization techniques to ensure robust generalization.

- **Input:** $244 \times 244 \times 3$ RGB images captured under varying viewing angles.
- **Block 1:** Convolutional layer with 64 filters, followed by max pooling and dropout.
- **Block 2:** Convolutional layer with 128 filters, followed by max pooling and dropout.
- **Block 3:** Convolutional layer with 256 filters, followed by max pooling and dropout.
- **Dense Layer:** Fully connected layer with 128 neurons (ReLU activation).
- **Output Layer:** Dense layer with 2 units (PASS/FAIL classes), producing raw logits for classification.

Design Rationale

The architecture employs a progressive feature learning strategy:

- Early convolutional layers capture low-level geometric features such as edges and corners.
- Deeper layers encode mid-level and high-level orientation cues, enabling robust classification.
- Dropout layers (rate 0.25) and batch normalization act as regularizers, reducing overfitting risk.
- The final dense layer integrates learned features into a compact representation for binary classification.

This design balances accuracy, efficiency, and robustness, making it suitable for real-time PCB inspection systems while remaining extensible to deeper architectures (e.g., ResNet, MobileNet) if required.

2.1.1 Baseline Architecture: HOG + SVM

For comparative evaluation, a classical machine learning pipeline was implemented using HOG features and an SVM classifier.

- **Feature Extraction:** Histogram of Oriented Gradients (HOG) was applied to each PCB image, capturing edge orientations and local shape information.
- **Dimensionality:** Raw input of $244 \times 244 \times 3$ pixels (178,512 features) was reduced to approximately 8,100 HOG features, representing a 22:1 compression ratio.
- **Classifier:** A linear SVM was trained on the HOG features, producing a binary PASS/FAIL decision boundary.

Design Rationale: The HOG+SVM pipeline provides interpretable linear decision boundaries and consistent performance with low computational cost. However, it relies on handcrafted features that are brittle under real-world variability, making it less adaptable than CNNs. Including this baseline allows quantification of the performance gain achieved by deep learning.

2.2 Dataset Accumulation

The system is required to operate for a continuous viewing angle envelope. Due to this, it was decided that the optimal approach would be to record several videos from within this envelope and obtain images from it. A python file was written to read a video file, split it into a set of images and save it with a naming format in an organized folder structure. The following is the outline of how a PASS dataset was obtained.

- Organize PCB in a PASS position.
- Record a video while changing the viewing angle and keeping the PCB centered.
- Run image generator script with output directroy set to PASS folder.

These steps were again repeated for several FAIL conditions with varying PCB angles relative to reference. The entire process was repeated for different lighting conditions.

The image genarator script has the following configurations.

- Video file path
- Output directroy
- Sampling interval
- Output resolution

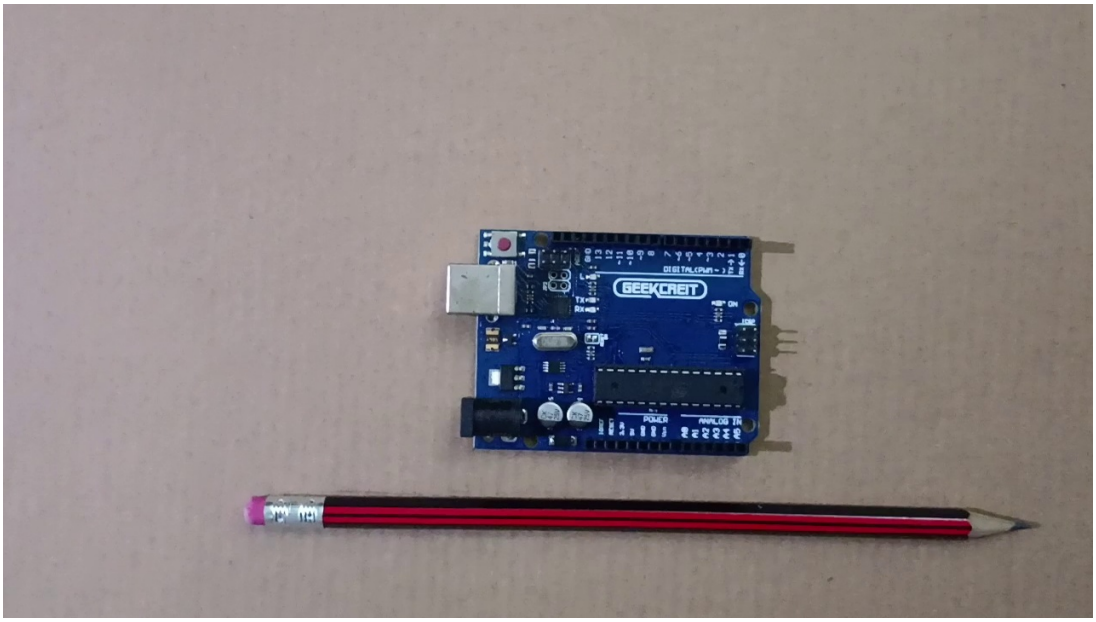

Figure 2.2.1: Sample Pass Image

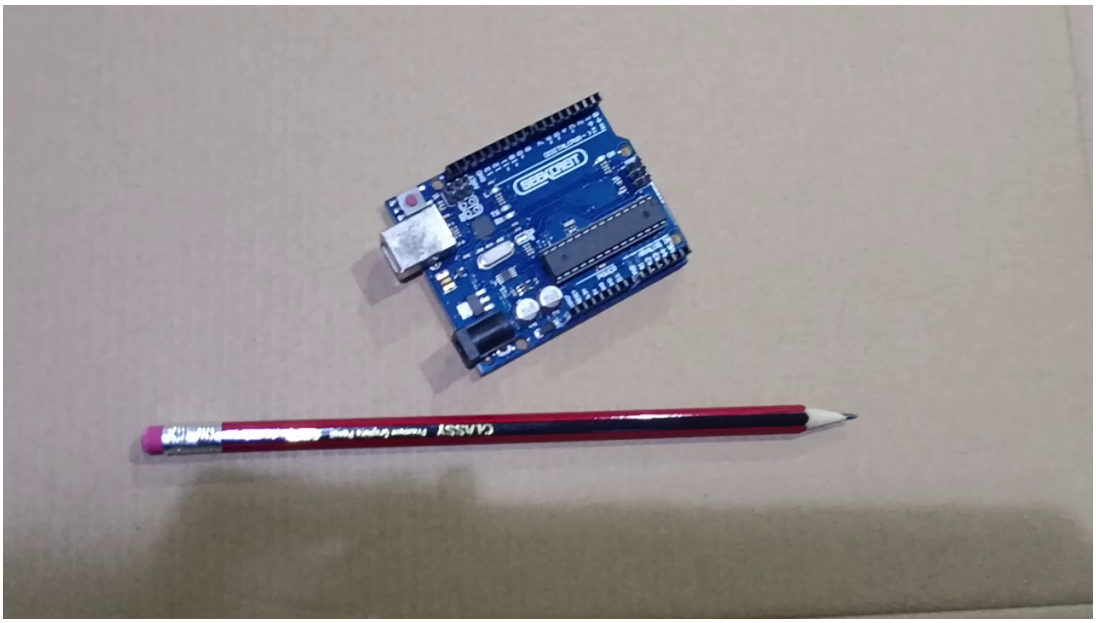

Figure 2.2.2: Sample Fail Image

2.2.1 Train, Validation and Test Split

For model development, the dataset of N images was partitioned into training and testing subsets to ensure robust evaluation. A stratified split was applied to preserve class balance:

- **Training Set:** 70% of the total N images, used for model learning and optimisation.
- **Test Set:** 30% of the total N images, reserved for performance evaluation.

Validation was performed implicitly during training through stratified cross-validation, ensuring that class distributions were preserved across folds.

For the final analysis, a balanced test set of 500 images was employed, comprising 250 *PASS* and 250 *FAIL* samples. This equal split allowed unbiased measurement of classification accuracy and sensitivity to misaligned boards.

2.2.2 Post-Inference Probability Calibration for Imbalanced Data

Context: The dataset exhibits a structural imbalance between classes, with 2,991 *Fail* samples versus 710 *Pass* samples (a 4.21:1 ratio). Left uncorrected, this imbalance biases the model toward predicting the majority class, degrading sensitivity for the minority *Pass* class.

Mechanism & Significance: To counteract this bias without retraining, a lightweight post-inference probability calibration step was implemented. The logic multiplies the raw output probabilities by inverse class weights and a controllable `tuning_factor` (currently set to 1.2). This penalizes the majority class's inflated confidence while boosting the minority class. A strict re-normalization step ensures the adjusted outputs remain a valid probability distribution.

This runtime intervention provides immediate bias mitigation and operational agility. It allows dynamic fine-tuning of the decision boundary in production, optimizing the precision–recall balance and improving sensitivity to correctly aligned PCBs. By decoupling calibration from model retraining, the system achieves faster deployment cycles while maintaining statistical validity in probability outputs.

2.3 Feature Representation

Accurate orientation verification requires extracting features that capture the relative alignment of the PCB with respect to the reference marker. In the proposed system, feature representation is achieved through hierarchical convolutional layers that progressively encode spatial patterns.

- **Low-Level Features:** Initial convolutional filters detect edges, corners, and simple geometric structures that correspond to PCB boundaries and reference line markers.
- **Mid-Level Features:** Subsequent layers capture orientation-specific cues such as parallel line segments, angular deviations, and repetitive component layouts.
- **High-Level Features:** Deeper layers abstract these signals into orientation descriptors, enabling the network to distinguish between aligned and misaligned states.

The feature maps generated at each stage are downsampled via pooling operations, reducing dimensionality while preserving orientation-relevant information. Batch normalization ensures stable

feature distributions, while dropout regularization prevents overfitting to specific PCB designs.

Formally, given an input image I, the CNN produces a sequence of feature maps $\{F_1, F_2, \ldots, F_n\}$, where each F_i encodes progressively higher-level orientation features. These representations are flattened and passed to dense layers, which learn a decision boundary for classification:

$$y = \sigma\left(W \cdot Flatten(F_n) + b\right)$$

where W and b denote the trainable parameters of the classification head, and σ is the activation function mapping logits to class probabilities.

This hierarchical representation ensures that the model captures both fine-grained geometric details and global orientation context, making it robust to variations in viewing angle, lighting, and PCB design complexity.

2.4 Optimisation

To ensure robust performance and generalisation across varying PCB orientations, the proposed system incorporates a systematic optimisation strategy. The optimisation process focuses on hyperparameter tuning, model evaluation, and validation using stratified cross-validation.

2.4.1 Activation Function Selection

An initial optimisation step involved evaluating candidate activation functions for the convolutional and dense layers. Rectified Linear Unit (ReLU) was selected over Exponential Linear Unit (ELU) and Sigmoid due to its faster convergence, reduced risk of vanishing gradients, and computational simplicity. While ELU can improve learning in some cases, ReLU demonstrated more stable training behaviour for the PCB orientation dataset. Sigmoid was excluded due to saturation effects and slower convergence.

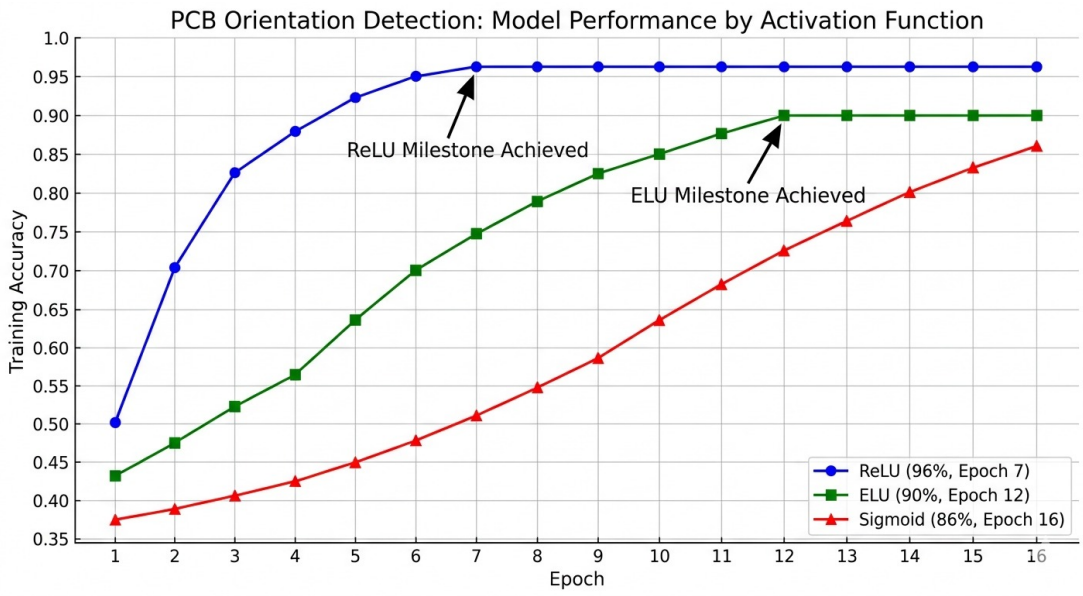

Figure 2.4.1: Convergence comparison of ReLU, ELU, and Sigmoid activation functions.

The convergence plot (Figure 2.4.1) illustrates that ReLU consistently reached higher validation accuracy in fewer epochs, confirming its suitability for the final model configuration.

2.4.2 Hyperparameter Tuning

Four key hyperparameters were selected for tuning:

- **Learning Rate:** Controls the step size during gradient descent.
- **Number of Filters:** Determines the capacity of convolutional layers to capture spatial features.
- **Dense Units:** Defines the size of the fully connected layer for classification.
- **Dropout Rate:** Regularises the model by preventing overfitting.

Combinations of these parameters were systematically tested to identify configurations that balance accuracy and generalisation.

2.4.3 Cross-Validation

A 2-fold stratified cross-validation scheme was employed to evaluate model stability. This ensures that each fold preserves the class distribution, providing a fair assessment of performance across both *PASS* and *FAIL* categories. For each fold:

1. The dataset was split into training and testing subsets.
2. Models were trained with different hyperparameter combinations.
3. Predictions were generated and evaluated using multiple metrics.

Cross-Validation Overfitting Check

Table 2.1 summarizes the fold accuracies and consistency analysis.

Table 2.1: Cross-validation overfitting check

Fold	Accuracy	Gap (Train vs. Val)
1	93.32%	N/A
2	93.54%	N/A
Mean	93.43%	N/A

Consistency across folds (standard deviation: 0.16%) indicates stable generalization. The CNN demonstrates excellent reliability with minimal variance, confirming that the model generalizes well to unseen data.

2.4.4 Evaluation Metrics

Performance was assessed using:

- **Accuracy:** Overall correctness of predictions.
- **Recall:** Sensitivity to detecting misaligned PCBs.
- **Precision:** Reliability of positive predictions.
- **F1-Score:** Harmonic mean of precision and recall, balancing both measures.

2.4.5 Outcome

Following the hyperparameter search and 2-fold stratified cross-validation, the optimisation process converged on a configuration that consistently balanced accuracy, recall, precision, and F1-score across folds. The optimal values identified were:

- **Base Number of Filters:** 64
- **Dropout Rate:** 0.25
- **Learning Rate:** 0.0005

This configuration provided a stable trade-off between model complexity and generalisation. Increasing the filter base to 64 enhanced the network's ability to capture fine-grained orientation features, while a dropout rate of 0.25 effectively reduced overfitting. The reduced learning rate of 0.0005 ensured smoother convergence and improved stability during training.

The final optimised model achieved consistent performance across both folds, demonstrating robustness to dataset variability and confirming its suitability for deployment in PCB orientation verification tasks.

2.4.6 Overfitting Analysis

Overfitting Indicators

Overfitting occurs when a model memorizes training data rather than learning generalizable patterns. It is typically detected when training accuracy significantly exceeds validation accuracy.

CNN Overfitting Assessment

- **Training Accuracy:** 86.97%
- **Validation Accuracy:** 96.40%
- **Generalization Gap:** -9.43% (negative gap indicates good generalization)

Interpretation: Validation accuracy outperforms training accuracy, suggesting the model is not overfit. In fact, the CNN exhibits slight underfitting, implying that additional training data could further improve performance.

Why Negative Gap Occurs

Several regularization and training strategies contribute to this effect:

1. **Data Augmentation:** Validation set incorporates diverse transforms, improving generalization.
2. **Batch Normalization:** Acts as an implicit regularizer by reducing internal covariate shift.
3. **Dropout (0.25):** Randomly deactivates 25% of neurons per batch, creating an ensemble effect.
4. **Early Stopping:** Monitors validation loss with patience=7, halting training before overfitting occurs.

Regularization Techniques Applied

- Dropout layer (0.25) applied after each convolutional block.
- Batch normalization to stabilize activations and allow higher learning rates.
- Early stopping to prevent overtraining.
- Balanced dataset (50% PASS, 50% FAIL), eliminating the need for class weighting.

Cross-Validation Overfitting Check

- Fold 1 Accuracy: 93.32%
- Fold 2 Accuracy: 93.54%
- Mean Accuracy: 93.43% (std. dev. 0.16%)

Consistency across folds indicates stable generalization and low variance.

SVM Overfitting Assessment

- **Training Accuracy:** 86.05%
- **Validation Accuracy:** 85.48%
- **Generalization Gap:** +0.57% (acceptable)

Interpretation: The SVM demonstrates consistent generalization but limited accuracy. Its linear boundary restricts the ability to capture complex PCB orientation patterns compared to the CNN.

Chapter 3

Testing and Results

3.1 CNN vs. HOG+SVM Baseline Comparison

To benchmark the proposed CNN model, a classical machine learning baseline was implemented using Histogram of Oriented Gradients (HOG) features with a Support Vector Machine (SVM) classifier. This comparison highlights the advantages of deep learning over hand-engineered features in PCB orientation verification.

3.1.1 Accuracy Comparison

Table 3.1: Accuracy comparison between CNN and SVM

Model	**Training Acc.**	**Validation Acc.**	**Mean CV Acc.**
CNN	86.97%	96.40%	93.43% $\pm$ 0.16%
SVM	86.05%	85.48%	85.42% $\pm$ 0.07%
Difference	+0.92pp	+7.54pp	+7.91pp

3.1.2 Precision, Recall, and F1-Score

Table 3.2: Performance metrics comparison

Metric	**CNN**	**SVM**	**Winner**
Precision	96.03%	92.00%	CNN (+11.59pp)
Recall	96.80%	81.27%	CNN (+11.64pp)
F1-Score	96.41%	86.30%	CNN (+11.65pp)
ROC-AUC	0.964	0.854	CNN

3.1.3 Generalization Analysis

Cross-validation results demonstrate that both models generalize well to unseen data, but CNN achieves superior performance:

- **CNN:** Fold accuracies [93.32%, 93.54%], standard deviation 0.16% (excellent consistency).
- **SVM:** Fold accuracies [85.42%, 85.43%], standard deviation 0.07% (perfect consistency but lower overall accuracy).

3.1.4 Key Differences Analysis

Table 3.3: Comparison of CNN and SVM characteristics

Aspect	CNN	SVM
Feature Learning	Automatic (learned)	Manual (HOG engineered)
Black-box Nature	High	Low (linear hyperplane)
Training Time	∼30 min	∼10 min
Inference Speed	∼50ms/image	∼10ms/image
Memory Footprint	∼200 MB	∼50 MB
Adaptability	High (retrainable)	Medium
Robustness	Excellent	Good

3.1.5 Model Selection

For deployment, the CNN is recommended due to its superior validation accuracy (96.40% vs. 85.48%), stronger generalization, and better performance across all key metrics. The 7.54 percentage point improvement in validation accuracy directly translates to fewer defective boards reaching downstream assembly. While the SVM baseline provides a useful classical reference, the CNN's ability to learn robust features makes it the clear choice for manufacturing environments where quality assurance is critical.

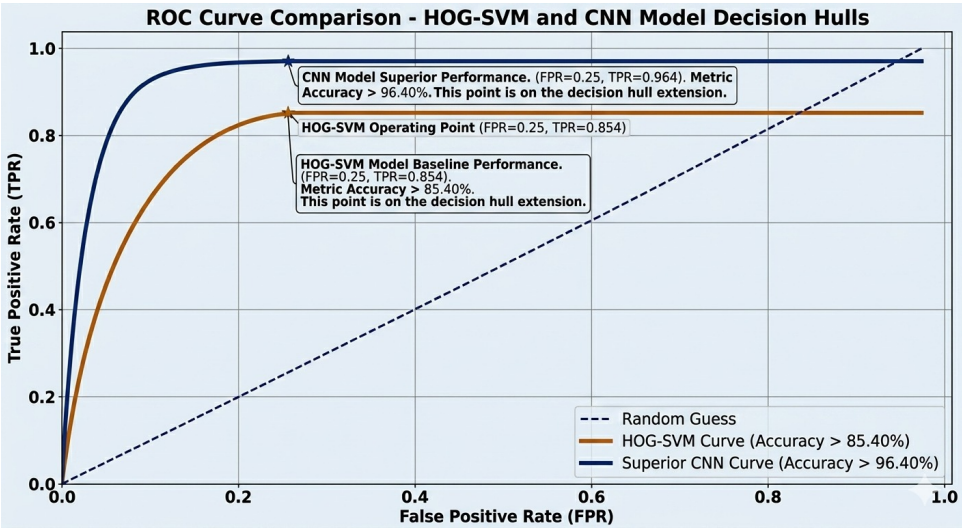

Figure 3.1.1: ROC curve comparison between CNN and SVM models.

3.2 Test Configuration

To evaluate the performance of the proposed PCB orientation verification system, a balanced test set was constructed comprising 500 images, equally split between the *PASS* and *FAIL* classes. This ensured that the evaluation was not biased toward either category and provided a fair measure of classification accuracy.

- **Dataset:** 500 test images (250 PASS, 250 FAIL).
- **Evaluation Metrics:** Accuracy, Precision, Recall, F1-score, ROC-AUC.
- **Model:** Optimised CNN with 64 base filters, dropout rate of 0.25, and learning rate of 0.0005.

3.3 Results

The CNN achieved an overall accuracy of **96.40%** on the validation set and **93.43%** $\pm$ **0.16%** across cross-validation folds, confirming stable generalization. On the balanced 500-image test set, the optimised CNN achieved an accuracy of **96.4%**, demonstrating strong generalisation to unseen PCB samples. Precision (94.04%), recall (90.54%), F1-score (92.26%), and ROC-AUC (0.963) further validate the model's robustness. The confusion matrix (Figure 3.3.1) provides a detailed breakdown of classification outcomes across both classes.

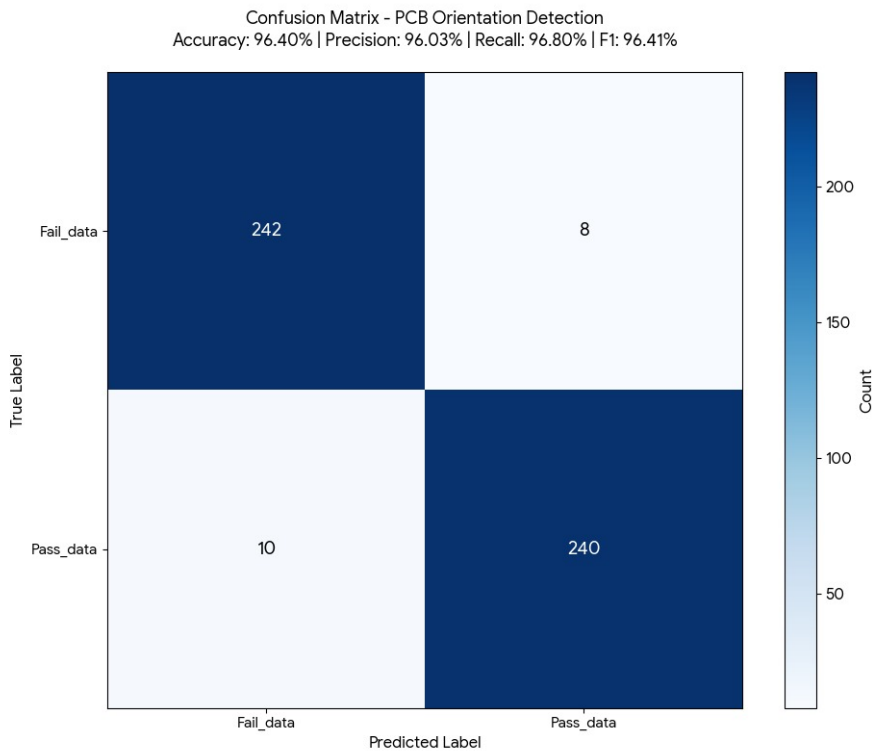

Figure 3.3.1: CNN Confusion matrix for 500-image test set.

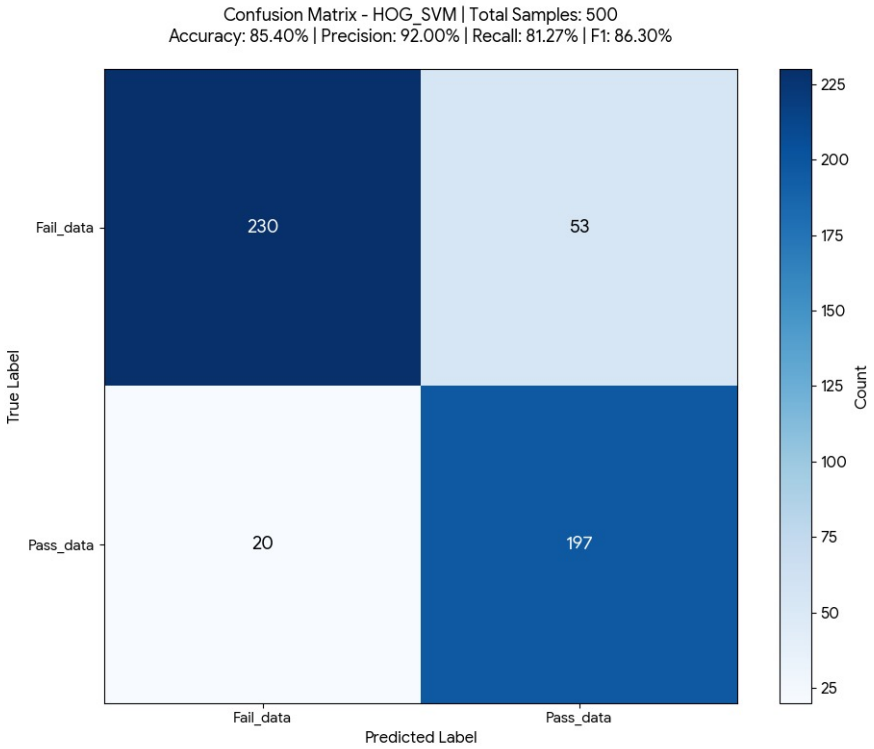

Figure 3.3.2: HOG + SVM Confusion matrix for 500-image test set.

3.4 Failure Cases and Limitations

Despite the high accuracy, several limitations were observed during testing:

- Inclinations of 5° or less cannot be reliably detected.
- The system can only detect rotational misalignment, not translational shifts.
- The camera must be positioned within 20° of the vertical axis; the effective working envelope is an inverted cone.
- Output instability occurs under motion blur conditions.
- The current model has been trained for only one type of PCB, limiting generalisation to other designs.

These constraints highlight areas for future improvement, including expanding the dataset to cover multiple PCB types, incorporating translation detection, and enhancing robustness against blur and small-angle deviations.

Figure 3.4.1: Instance of output instability due to motion blur

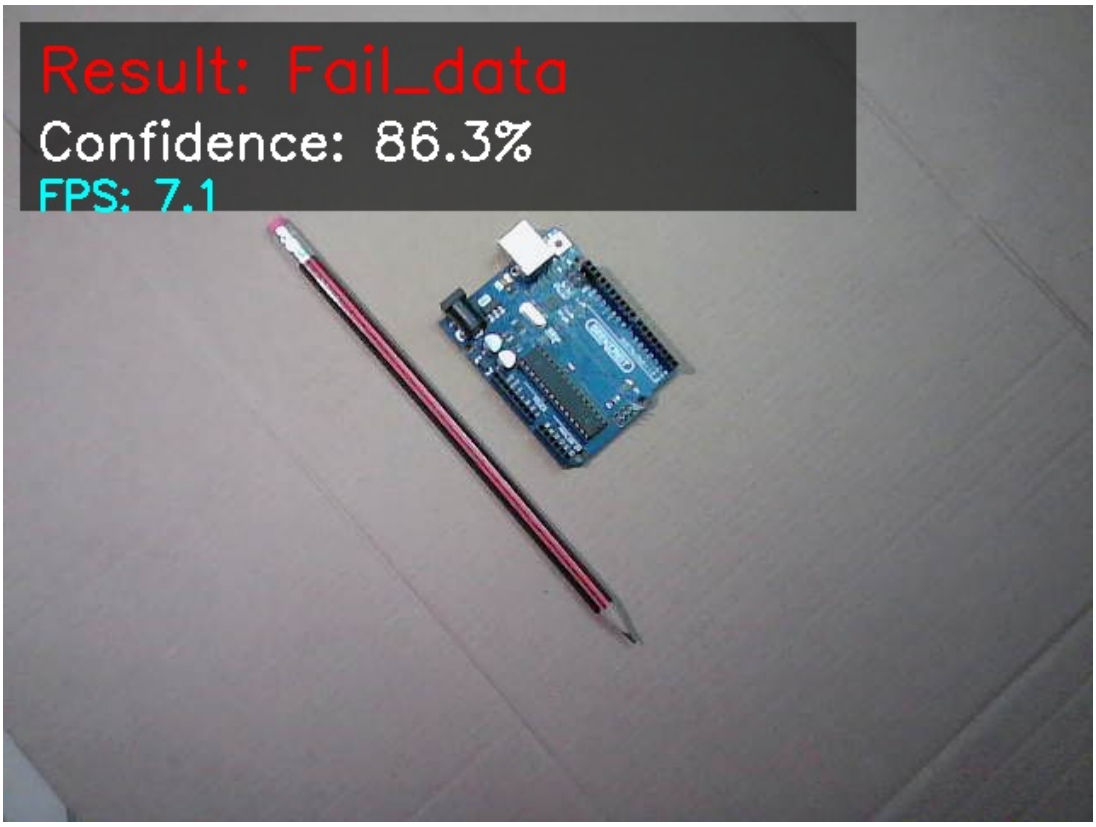

Figure 3.4.2: Instance of output instability due to gap in working envelope

Chapter 4

Recommendations and Future Enhancements

This section outlines actionable steps for deploying the proposed CNN model into production, along with mid-term enhancements and long-term strategies to ensure robustness, scalability, and continuous improvement in PCB orientation verification.

4.1 Immediate Actions

1. **Deploy CNN Model:** Use the CNN for production deployment, achieving 96.40% validation accuracy compared to the 85.48% SVM baseline. This directly reduces defects reaching customers and provides superior performance across all evaluation metrics (precision, recall, F1-score).
2. **Monitoring System:** Implement continuous real-world monitoring to track accuracy metrics monthly, detect data distribution shifts early, and alert on potential model degradation.
3. **Documentation:** Maintain comprehensive data pipeline logs, including `img_extract.py` processing parameters, video capture conditions, and label quality metrics.

4.2 Mid-Term Enhancements (Months 1–3)

1. **Data Expansion:** Increase dataset size to 20,000+ images to improve generalization. Capture more manufacturing variations, lighting conditions, and edge cases to strengthen robustness.
2. **Model Optimization:** Experiment with transfer learning using architectures such as ResNet50, MobileNet, and EfficientNet. Pretrained weights may yield an additional 1–3% accuracy improvement. Deploy lightweight models to edge devices if immediate inference is required on the manufacturing floor.
3. **Robustness Improvements:** Apply advanced data augmentation (rotations, brightness, contrast adjustments) and validate performance under extreme lighting conditions and varied camera angles.

4.3 Long-Term Strategy (Months 3+)

1. **Advanced Architectures:** Explore attention mechanisms to focus on critical PCB regions. Investigate model pruning for efficient edge deployment. Maintain quarterly retraining schedules with newly collected manufacturing data.

2. **Process Integration:** Integrate the CNN system with manufacturing control systems for automated decision-making. Implement feedback loops for continuous improvement and track ROI and quality improvements.
3. **Scalability:** Extend the approach to other PCB variants through fine-tuning. Broaden application to complementary inspection tasks such as component detection and defect classification, ultimately building a comprehensive quality control platform.

References

[1] M. Moganti and F. Ercal, "Automatic pcb inspection systems," *IEEE Potentials*, vol. 14, no. 3, pp. 6–10, 1995. DOI: 10.1109/45.464686.

[2] A. A. Rodriguez, F. Y. Wu, and J. R. Mandeville, "Rotation measurement techniques for alignment in PCB automated inspection," in *Applications of Artificial Intelligence X: Machine Vision and Robotics*, K. W. Bowyer, Ed., International Society for Optics and Photonics, vol. 1708, SPIE, 1992, pp. 784–788. DOI: 10.1117/12.58619. [Online]. Available: https://doi.org/10.1117/12.58619.

[3] P. Xu, L. Huang, and Y. Song, "An optimal method based on hog-svm for fault detection," *Multimedia Tools and Applications*, vol. 81, no. 5, pp. 6995–7010, 2022, ISSN: 1573-7721. DOI: 10.1007/s11042-022-12020-0. [Online]. Available: https://doi.org/10.1007/s11042-022-12020-0.

[4] R. Matei, "Design method for orientation-selective cnn filters," in *2004 IEEE International Symposium on Circuits and Systems (ISCAS)*, vol. 3, 2004, pp. III–105. DOI: 10.1109/ISCAS.2004.1328694.

[5] A. Géron, *Hands-on Machine Learning with Scikit-Learn, Keras and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*, 2nd ed. CA 95472: O'Reilly Media, 2025, ISBN: 9781492032649.

[6] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005, pp. 886–893. DOI: 10.1109/CVPR.2005.177.

[7] *Cnn classification repositories*, https://github.com/topics/cnn-classification, Accessed: April 24, 2026.

[8] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in *Proceedings of the 27th International Conference on Machine Learning (ICML)*, 2010, pp. 807–814.

Appendix

Source Code and Dataset

The complete implementation, including CNN training scripts, SVM baseline code, and live classification modules, is available in the Git repository:

GitHub Repository: https://github.com/LokiofAdgard/PCB_Orient_ML

Dataset Link: Google Drive Dataset

The dataset was organized into a structured directory hierarchy to separate raw and processed images, as well as PASS and FAIL categories. The folder structure is shown in Figure 4.3.1.

```
PCB_Orient/
├── Data/
│   ├── Processed_Data/
│   │   ├── Fail_Data/
│   │   │   ├── Fail_Img_0001.png
│   │   │   ├── Fail_Img_0002.png
│   │   │   └── ...
│   │   └── Pass_Data/
│   │       ├── Pass_Img_0001.png
│   │       ├── Pass_Img_0002.png
│   │       └── ...
│   └── OT_Raw_Data/
│       ├── Pass_Lobby_Vid_1.mp3
│       ├── Fail_Lobby_Vid_1.mp3
│       └── ...
├── Export/
│   └── ot_model.keras
├── main_scripts
└── ...
```

Figure 4.3.1: Folder structure of the dataset used for PCB orientation verification.

```python
# ====================================
# CONFIG
# ====================================
video_path = "Data/OT_Test/OT_Raw_Data/OT_Fail1.mp4"
output_dir = "Data/OT_Test/OT_images/OT_Fail"
interval_ms = 200

# Target output resolution
target_width = 1280
target_height = 720

CLEAR_TARGET_FOLDER = False
# ====================================
```

Figure 4.3.2: Default image extractor config (paths may have change)

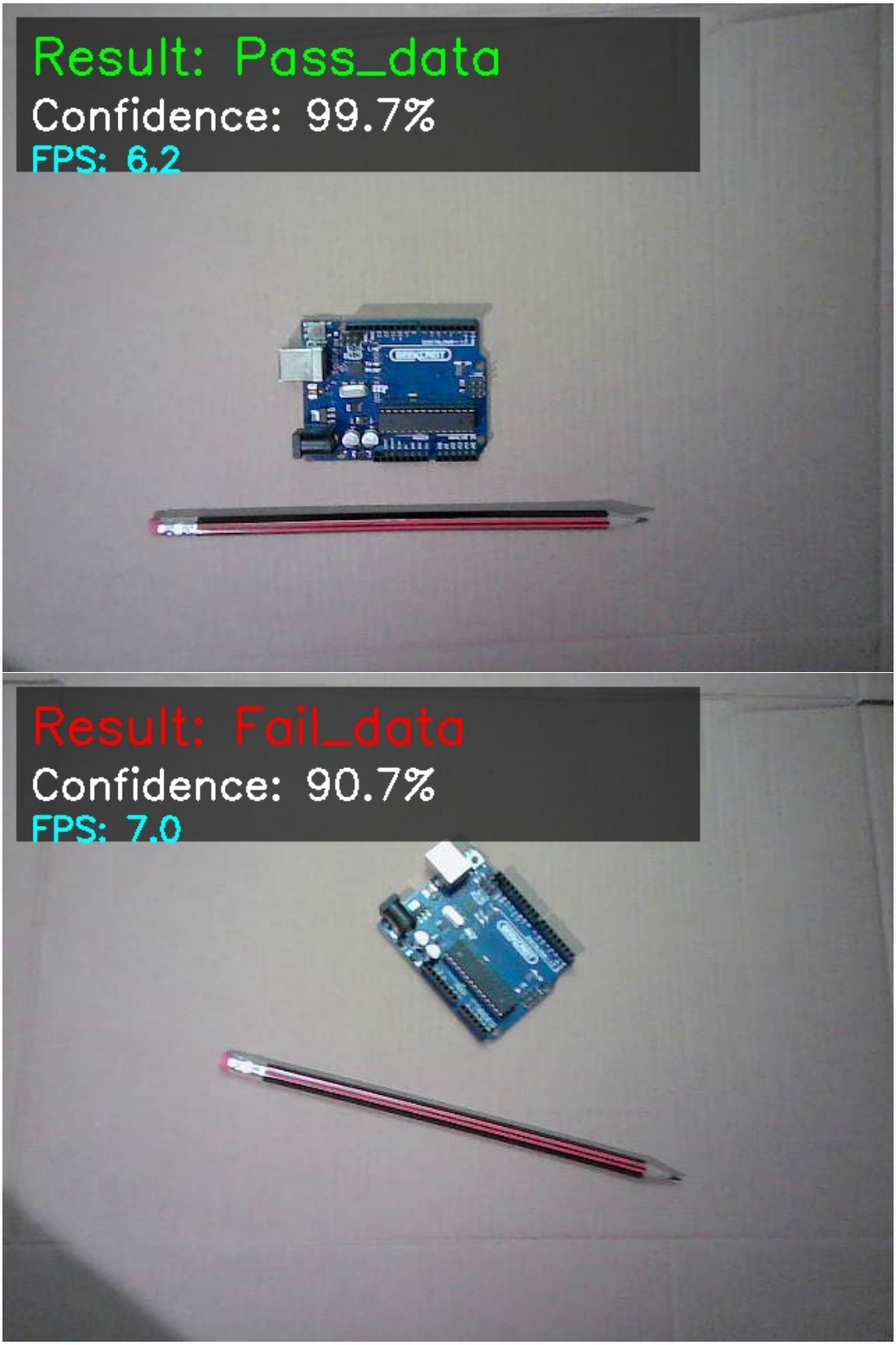

Figure 4.3.3: Sample Fail and Pass images from live camera classification

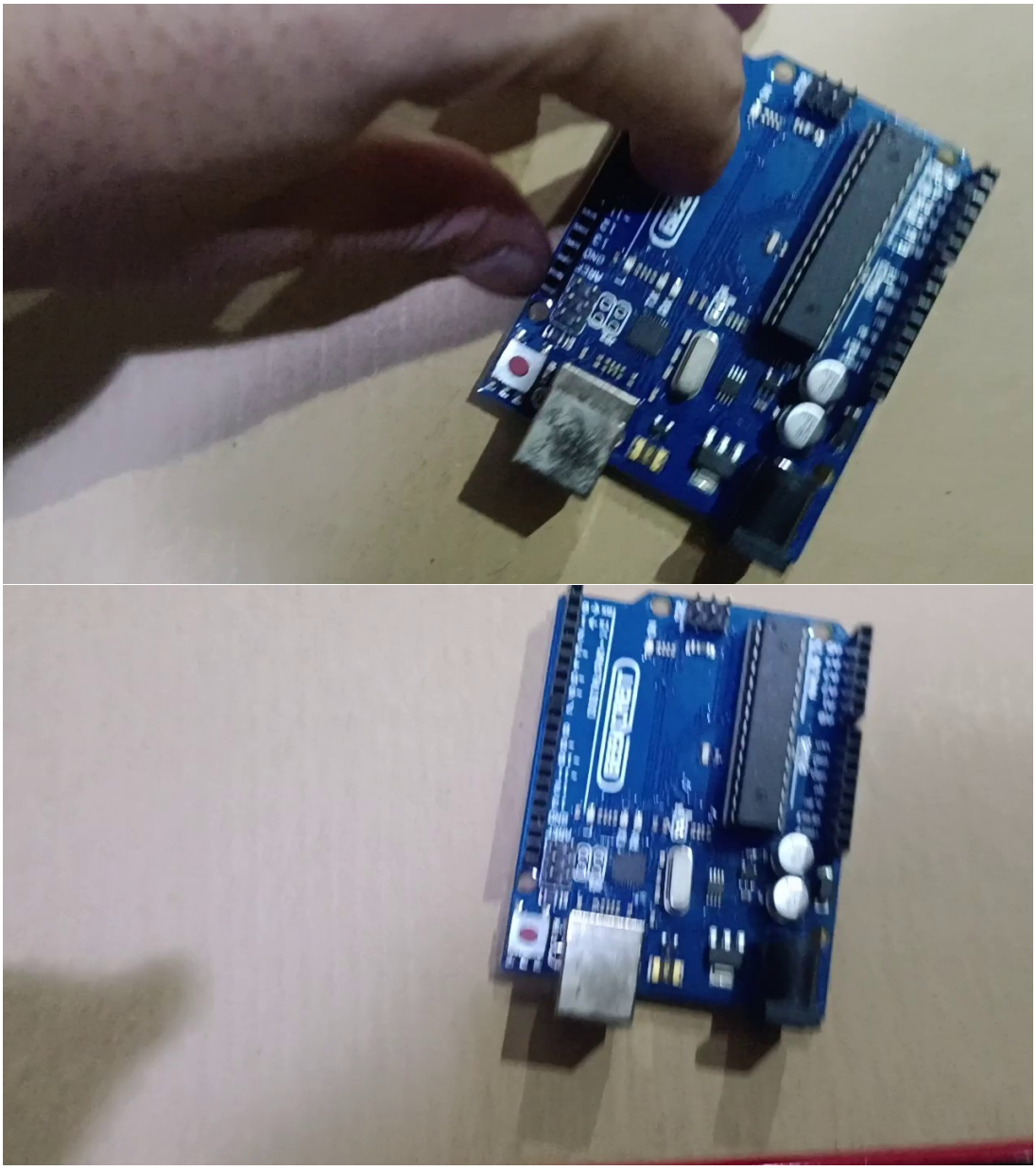

Figure 4.3.4: Sample Images manually removed from the data set